ADVANCE REACT JS

SUNITA D RATHOD

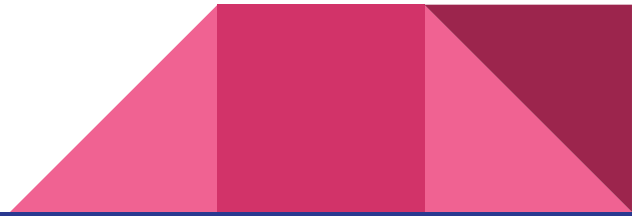
REACT FLUX

Flux is an application architecture that Facebook uses internally for building the client-side web application with React.

It is a kind of architecture that complements React as view and follows the concept of Unidirectional Data Flow model.

It is useful when the project has dynamic data, and we need to keep the data updated in an effective manner.

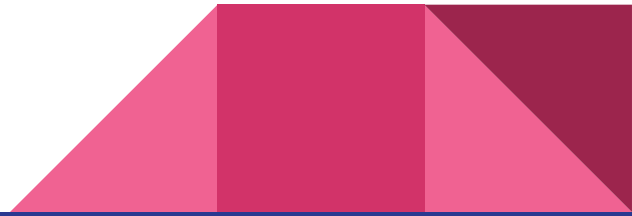
It reduces the runtime errors.



REACT FLUX

Flux applications have three major roles in dealing with data:

1. Dispatcher
2. Stores
3. Views (React components)



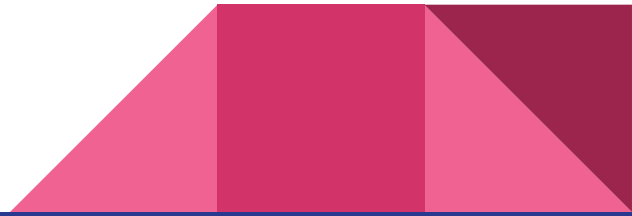
REACT FLUX

Flux controller-views (views) found at the top of the hierarchy.

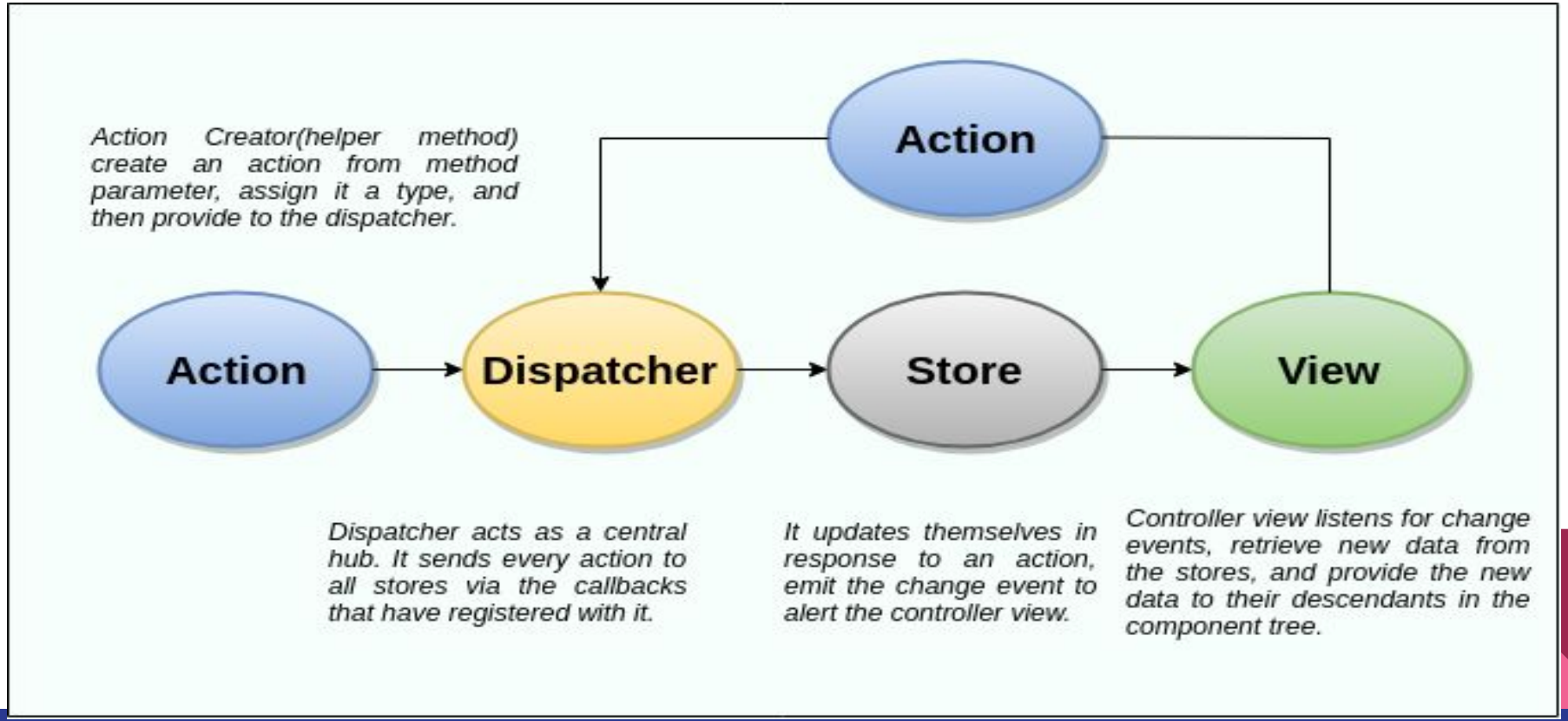
It retrieves data from the stores and then passes this data down to their children.

Additionally, action creators - dispatcher helper methods used to describe all changes that are possible in the application.

It can be useful as a fourth part of the Flux update cycle.



Structure and Data Flow

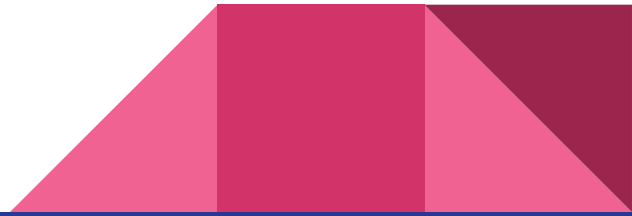


In Flux application, data flows in a single direction(unidirectional).

This data flow is central to the flux pattern.

The dispatcher, stores, and views are independent nodes with inputs and outputs.

The actions are simple objects that contain new data and type property.



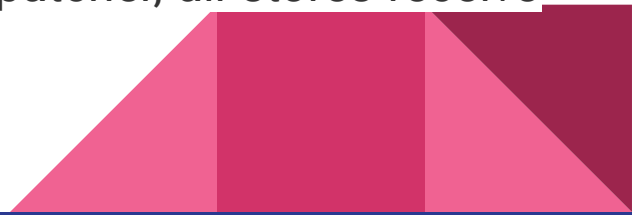
Dispatcher

It is a central hub for the React Flux application and manages all data flow of your Flux application.

It is a registry of callbacks into the stores. It has no real intelligence of its own, and simply acts as a mechanism for distributing the actions to the stores.

All stores register itself and provide a callback. It is a place which handled all events that modify the store.

When an action creator provides a new action to the dispatcher, all stores receive that action via the callbacks in the registry.



Dispatcher

The dispatcher's API has five methods. These are:

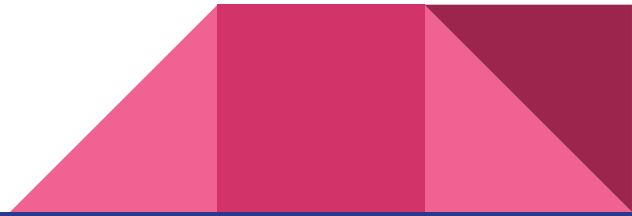
SN	Methods	Descriptions
1.	register()	It is used to register a store's action handler callback.
2.	unregister()	It is used to unregisters a store's callback.
3.	waitFor()	It is used to wait for the specified callback to run first.
4.	dispatch()	It is used to dispatches an action.
5.	isDispatching()	It is used to checks if the dispatcher is currently dispatching an action.

Stores

It primarily contains the application state and logic.

It is similar to the model in a traditional MVC.

It is used for maintaining a particular state within the application, updates themselves in response to an action, and emit the change event to alert the controller view.

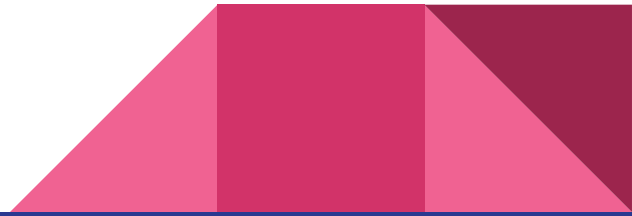


Views

It is also called as controller-views.

It is located at the top of the chain to store the logic to generate actions and receive new data from the store.

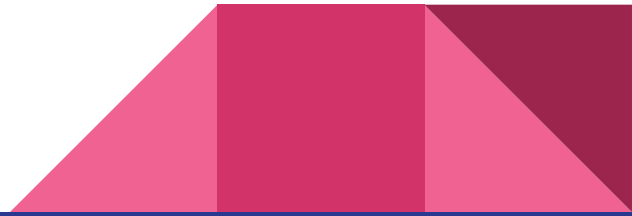
It is a React component listen to change events and receives the data from the stores and re-render the application.



Actions

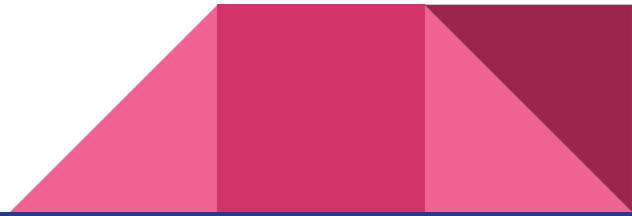
The dispatcher method allows us to trigger a dispatch to the store and include a payload of data, which we call an action.

It is an action creator or helper methods that pass the data to the dispatcher.



Advantage of Flux

- It is a unidirectional data flow model which is easy to understand.
- It is open source and more of a design pattern than a formal framework like MVC architecture.
- The flux application is easier to maintain.
- The flux application parts are decoupled.



MVC Architecture

MVC stands for Model View Controller. It is an architectural pattern used for developing the user interface.

It divides the application into three different logical components: the Model, the View, and the Controller.

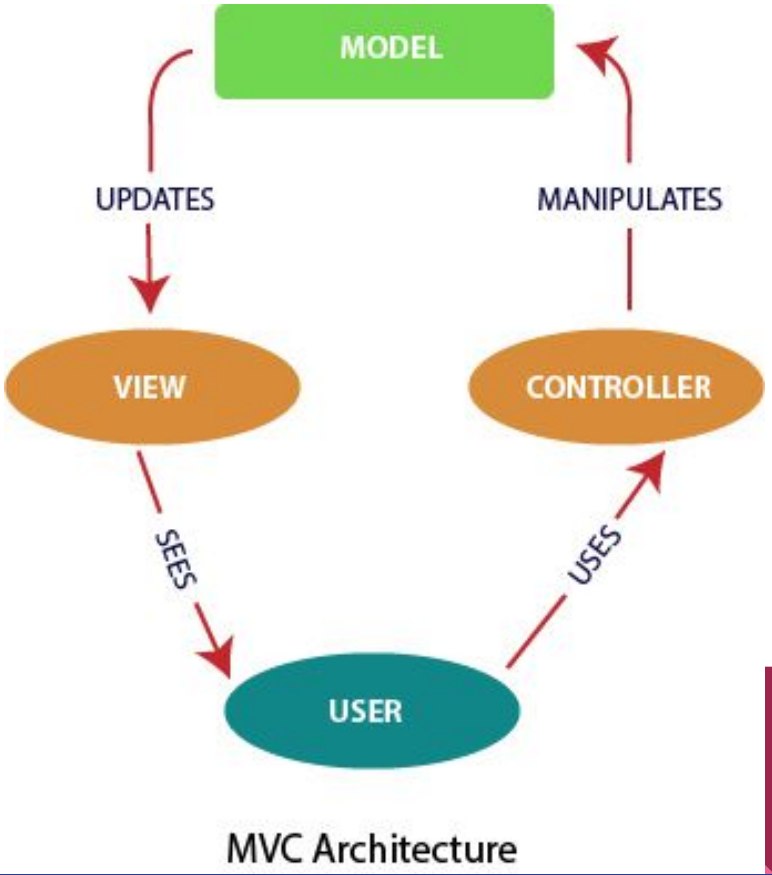
It is first introduced in 1976 in the Smalltalk programming language. In MVC, each component is built to handle specific development aspect of an application.

It is one of the most used web development frameworks to create scalable projects.

MVC Architecture

The MVC architecture contains the three components. These are:

- **Model:** It is responsible for maintaining the behavior and data of an application.
- **View:** It is used to display the model in the user interface.
- **Controller:** It acts as an interface between the Model and the View components. It takes user input, manipulates the data(model) and causes the view to update.



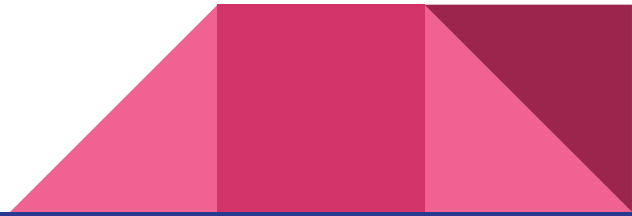
MVC Architecture

React Redux

Redux is an open-source JavaScript library used to manage application state.

React uses Redux for building the user interface.

It was first introduced by **Dan Abramov** and **Andrew Clark** in **2015**.



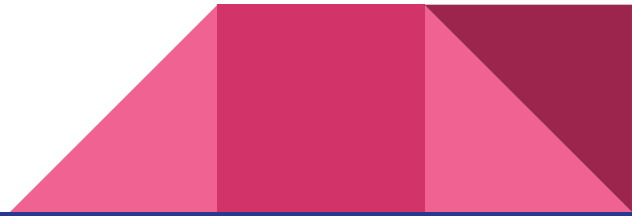
React Redux

React Redux is the official React binding for Redux.

It allows React components to read data from a Redux Store, and dispatch **Actions** to the **Store** to update data.

Redux helps apps to scale by providing a sensible way to manage state through a unidirectional data flow model. React Redux is conceptually simple.

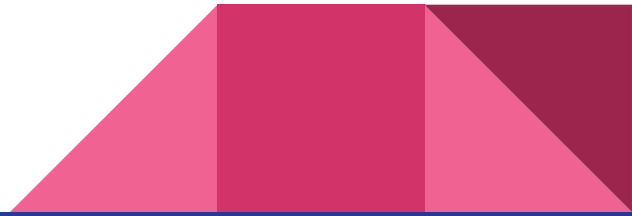
It subscribes to the Redux store, checks to see if the data which your component wants have changed, and re-renders your component.



React Redux

Redux was inspired by Flux. Redux studied the Flux architecture and omitted unnecessary complexity.

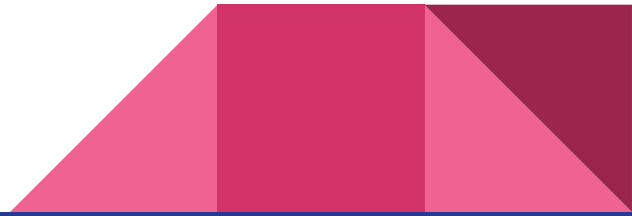
- Redux does not have Dispatcher concept.
- Redux has an only Store whereas Flux has many Stores.
- The Action objects will be received and handled directly by Store.



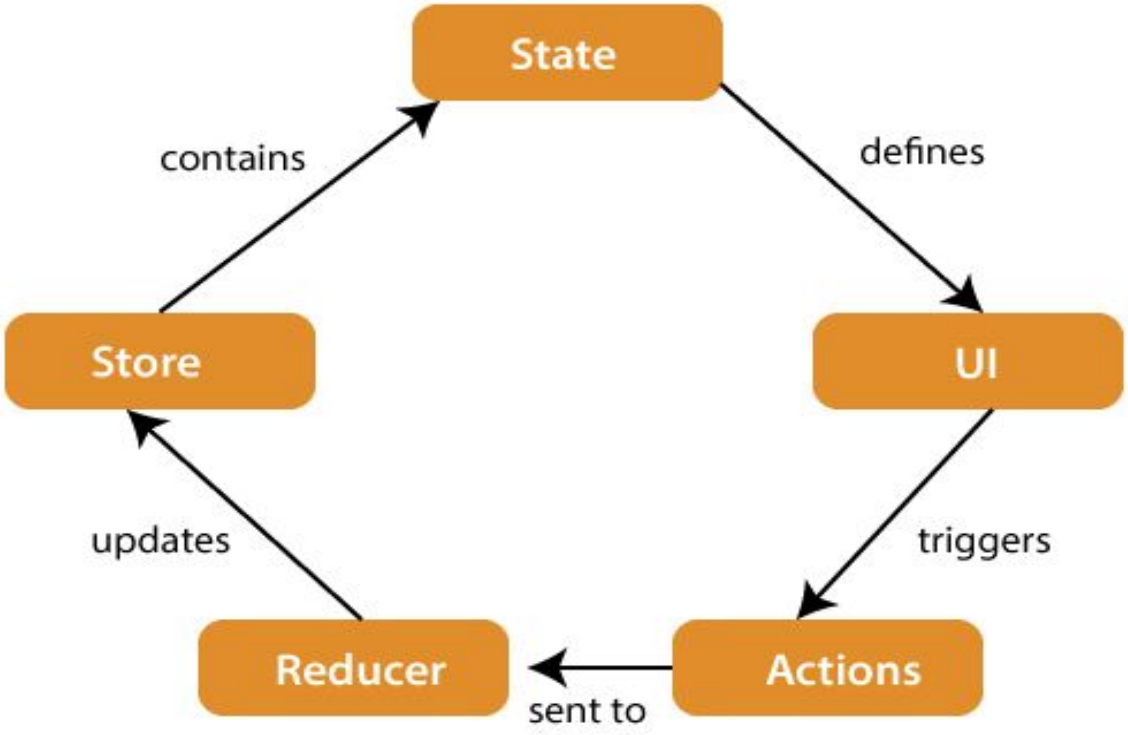
Why use React Redux?

The main reason to use React Redux are:

- React Redux is the official **UI bindings** for react Application. It is kept up-to-date with any API changes to ensure that your React components behave as expected.
- It encourages good 'React' architecture.
- It implements many performance optimizations internally, which allows to components re-render only when it actually needs.



Redux Architecture



Redux Architecture

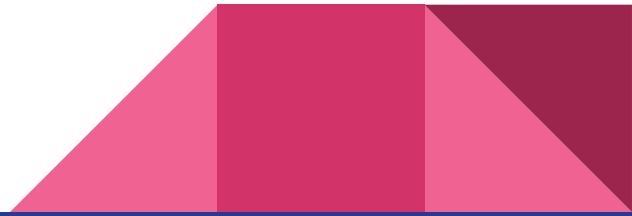
The components of Redux architecture are explained below.

STORE:

A Store is a place where the entire state of your application lists.

It manages the status of the application and has a dispatch(action) function.

It is like a brain responsible for all moving parts in Redux.



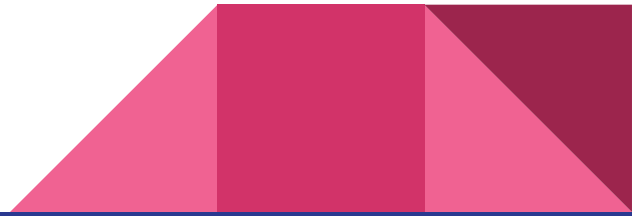
Redux Architecture

ACTION:

Action is sent or dispatched from the view which are payloads that can be read by Reducers.

It is a pure object created to store the information of the user's event.

It includes information such as type of action, time of occurrence, location of occurrence, its coordinates, and which state it aims to change.

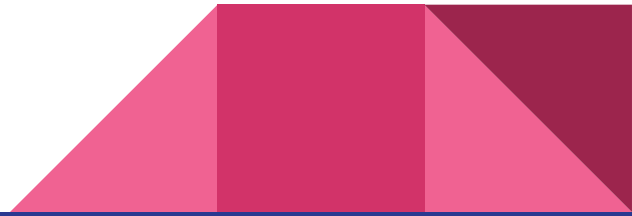


Redux Architecture

REDUCER:

Reducer read the payloads from the actions and then updates the store via the state accordingly.

It is a pure function to return a new state from the initial state.

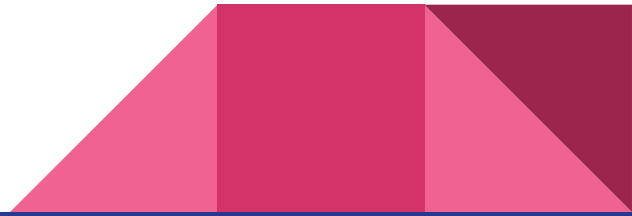


Redux Installation

Requirements: React Redux requires React 16.8.3 or later version.

To use React Redux with React application, you need to install the below command.

```
$ npm install redux react-redux --save
```



MVC vs FLUX vs REDUX

PARAMETERS	MVC	FLUX	REDUX
ARCHITECTURE	Architectural design pattern for developing UI	Application architecture designed to build client - side web apps	Open source JavaScript library used for creating the UI
DATA FLOW DIRECTION	Bi-directional flow	Unidirectional flow	Unidirectional flow
SINGLE OR MULTIPLE STORES	No concept of store	Includes multiple stores	Includes single store
BUSINESS LOGIC RESIDES	Controller handles entire logic	Store handles all logic	Reducer handles all logic

MVC vs FLUX vs REDUX

PARAMETERS	MVC	FLUX	REDUX
DEBUGGING	Debugging is difficult due to bi directional flow	Ensures simple debugging with the dispatcher	Single store makes debugging lot easier
USAGE	In both client & server side frameworks	Client-side framework	Client-side framework
SUPPORT	Front-end frameworks like Angular JS, Ember, Backbone, Sprout & Knockout Back-end frameworks like Spring Ruby on Rails, Django, Meteor	Front-end frameworks like React, AngularJS, Vue.js & polymer	Front-end frameworks like React, AngularJS, Vue.js Ember, Backbone.js, Meteor & polymer